BriefScope AI

AI Technical Brief Analyzer - Project Specification

1. Project objective

BriefScope AI is a full-stack platform that analyses technical briefs, freelance opportunities, job descriptions or client requests using AI.

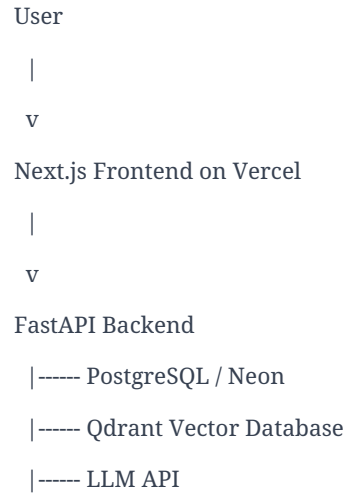
The user pastes a technical brief and the system returns structured outputs: project scope, required skills, technical risks, client questions, effort estimate, suggested daily rate, implementation plan and a proposal draft.

The project is designed as a production-ready portfolio project for a Full-Stack Engineer profile, showing practical experience with React, Next.js, TypeScript, Python, FastAPI, PostgreSQL, Neon, Qdrant, LLM APIs, Docker, Vercel and REST APIs.

2. Technology stack

Layer	Technology	Purpose
Frontend	Next.js, React, TypeScript	Dashboard, forms, detail pages and frontend routing
Backend	Python, FastAPI, Pydantic	REST APIs, AI orchestration, validation and business logic
Database	PostgreSQL on Neon	Main source of truth for briefs, analyses and proposals
Vector database	Qdrant	Embeddings, semantic search and similar brief matching
AI layer	LLM APIs and Embeddings	Brief analysis, proposal generation and vector search
DevOps	Docker, Docker Compose	Local development and backend containerisation
Deployment	Vercel, Railway/Render/Fly.io	Frontend production deploy and backend production deploy

3. High-level architecture



PostgreSQL/Neon is the main database and source of truth. Qdrant works alongside it as the vector database for semantic search and similarity matching.

4. Main user flow

- User creates a new brief from a job post, freelance request, client email or technical document.
- The frontend sends the brief to the FastAPI backend using REST APIs.
- The backend saves the brief in PostgreSQL on Neon.
- User clicks Analyze Brief.
- FastAPI sends the brief text to an LLM API and requests structured JSON output.
- The backend saves the AI analysis in PostgreSQL.
- The backend generates embeddings for the brief and stores them in Qdrant.
- The frontend displays summary, required skills, risks, questions, effort estimate, suggested rate and proposal draft.

5. MVP features

5.1 Dashboard

The dashboard shows all analysed and non-analysed briefs.

- Title
- Client or company name
- Source type
- Status
- Risk level
- Complexity
- Estimated effort
- Created date

Statuses: New, Analysed, Proposal Drafted, Archived.

5.2 Create brief

Form fields:

- Title
- Client name
- Source type
- Brief text

Source types: LinkedIn Job, Upwork Job, Client Email, Tender / RFP, Internal Project, Other.

5.3 AI brief analysis

The AI analysis is the core feature. It takes the brief text and returns structured data that can be stored and displayed.

AI output fields:

- Summary
- Required skills
- Nice-to-have skills
- Technical scope
- Main deliverables
- Missing information
- Risks
- Questions to ask
- Complexity
- Estimated effort
- Suggested daily rate
- Implementation plan

```
{
```

```
  "summary": "The client needs a full-stack dashboard with AI-powered analysis.",
  "required_skills": ["React", "Python", "FastAPI", "PostgreSQL"],
  "risks": ["Unclear authentication requirements", "No deployment strategy mentioned"],
  "complexity": "Medium",
  "estimated_effort": "12-18 days",
  "suggested_daily_rate": "EUR 230-280/day"
}
```

5.4 Proposal generator

After the analysis, the system generates a short proposal and a more technical version.

- Short proposal
- Technical proposal
- Questions for the client
- Suggested next step

5.5 Similar brief search with Qdrant

When a brief is analysed, the backend creates an embedding and stores it in Qdrant. This allows the platform to find similar briefs later.

Output:

- Similar brief title
- Similarity score
- Previous proposal
- Previous estimated effort
- Previous risk level

6. Non-MVP features

These features should not be built in the first version. They can be added later.

- Advanced authentication
- Payments
- Team management
- PDF upload
- Chrome extension
- Email integration
- Calendar reminders
- Multi-user permissions

7. Data model

7.1 briefs table

```
id
title
client_name
source_type
brief_text
status
risk_level
```

- complexity
- estimated_effort
- created_at
- updated_at

7.2 analyses table

- id
- brief_id
- summary
- required_skills
- nice_to_have_skills
- technical_scope
- deliverables
- missing_information
- risks
- questions
- complexity
- estimated_effort
- suggested_daily_rate
- implementation_plan
- created_at

7.3 proposals table

- id
- brief_id
- short_proposal
- technical_proposal
- client_questions
- next_step
- created_at

7.4 vector_metadata table

- id
- brief_id
- qdrant_point_id
- embedding_model
- created_at

8. Qdrant design

Qdrant collection name: brief_embeddings

Payload example:

```
{
  "brief_id": "uuid",
  "title": "Senior PHP Developer role",
  "source_type": "LinkedIn Job",
  "skills": ["PHP", "Laravel", "React", "SQL"],
  "risk_level": "Medium",
  "complexity": "Medium"
}
```

Qdrant use cases: semantic search, similar briefs, CV/profile matching and future RAG features.

9. Backend API endpoints

9.1 Health

GET /health

9.2 Briefs

GET /briefs

POST /briefs

GET /briefs/{brief_id}

PUT /briefs/{brief_id}

DELETE /briefs/{brief_id}

9.3 Analysis

POST /briefs/{brief_id}/analyze

GET /briefs/{brief_id}/analysis

9.4 Proposal

POST /briefs/{brief_id}/generate-proposal

GET /briefs/{brief_id}/proposal

9.5 Semantic search

POST /briefs/{brief_id}/similar

POST /search/semantic

10. Frontend pages

Page	Purpose	Main components
/	Landing page with project intro and CTA	Hero, CTA, feature cards
/dashboard	List all briefs	BriefCard, StatusBadge,

		RiskBadge, SearchInput
/briefs/new	Create a new brief	BriefForm, SourceTypeSelect
/briefs/[id]	Brief detail and analysis	AnalysisResult, SkillTag, ProposalBox, SimilarBriefs

11. Repository structure

```
briefscope-ai/
  frontend/
    app/
    components/
    lib/
    types/
    package.json
    next.config.js

  backend/
    app/
      main.py
      api/
        briefs.py
        analysis.py
        proposals.py
        search.py
      models/
        brief.py
        analysis.py
        proposal.py
      schemas/
        brief_schema.py
        analysis_schema.py
        proposal_schema.py
      services/
        llm_service.py
        embedding_service.py
        qdrant_service.py
        brief_analyzer.py
      db/
```

database.py

config.py

Dockerfile

requirements.txt

docker-compose.yml

README.md

12. Environment variables

Backend

```
DATABASE_URL=
QDRANT_URL=
QDRANT_API_KEY=
LLM_API_KEY=
LLM_MODEL=
EMBEDDING_MODEL=
FRONTEND_URL=
```

Frontend

```
NEXT_PUBLIC_API_URL=
```

13. Roadmap

Phase	Goal	Main tasks
Phase 1 - Setup	Frontend, backend, database and Docker running	Create repo, setup Next.js, setup FastAPI, setup PostgreSQL/Neon, setup Qdrant, GET /health
Phase 2 - CRUD	Save and read briefs	Brief model, POST /briefs, GET /briefs, dashboard, detail page
Phase 3 - AI analysis	Analyse briefs with AI	LLM service, structured JSON prompt, save analysis, display analysis
Phase 4 - Proposal generator	Generate proposal drafts	Prompt proposal, save proposal, display in UI
Phase 5 - Qdrant	Semantic search	Create embeddings, save in Qdrant, endpoint similar briefs
Phase 6 - Deploy	Production deployment	Vercel frontend, Railway/Render backend, Neon DB, Qdrant Cloud, CORS, README

14. First milestone

The first concrete milestone is to have the basic system running locally:

- Next.js frontend running on localhost:3000

- FastAPI backend running on localhost:8000
- PostgreSQL connected
- Qdrant running locally
- GET /health working
- POST /briefs working
- GET /briefs working

15. CV positioning after completion

Project title for CV:

BriefScope AI - Python / FastAPI / PostgreSQL / Neon / Qdrant / Next.js / React / TypeScript / LLM APIs / Docker

CV bullets:

- Built a full-stack AI platform to analyse technical briefs, extract scope, risks, missing requirements and implementation plans.
- Developed a Python/FastAPI backend with PostgreSQL/Neon, structured REST APIs and Docker-based deployment.
- Integrated LLM APIs and Qdrant vector search to classify briefs, generate proposal drafts and retrieve semantically similar projects.
- Built a React/Next.js dashboard in TypeScript and deployed the frontend on Vercel.